



Week 11 — Advanced Model Evaluation

সম্পূর্ণ Concept Guide

১. Confusion Matrix — ভুলগুলো ভেঙে দেখা

এটা কী?

Confusion Matrix হলো একটা table যেটা দেখায় তোমার model কোথায় সঠিক আর কোথায় ভুল করছে।

ধরো তুমি একটা model বানিয়েছ যেটা বলবে কোনো রোগী **cancer** আছে কি নেই।

Model predict করতে পারে দুটো জিনিস: **Positive (cancer আছে)** বা **Negative (cancer নেই)**। Reality তেও দুটো অবস্থা আছে।

এই দুটো মিলিয়ে ৪টা situation তৈরি হয়:

	Model বলল Positive	Model বলল Negative
আসলে Positive →	TP	FN
আসলে Negative →	FP	TN

চারটা **Term** একদম সহজে:

True Positive (TP)

Model বলল "cancer আছে" — আসলেও আছে। সঠিক!

True Negative (TN)

Model বলল "cancer নেই" — আসলেও নেই। সঠিক!

False Positive (FP) — Type I Error

Model বলল "cancer আছে" — কিন্তু আসলে নেই। ভুল alarm! মানে model অতিরিক্ত সন্দেহপ্রবণ।

False Negative (FN) — Type II Error

Model বলল "cancer নেই" — কিন্তু আসলে আছে। বিপজ্জনক ভুল! মানে model রোগটা miss করে গেছে।

কোনটা বেশি ক্ষতিকর?

এটা **context** এর উপর নির্ভর করে:

- Cancer detection এ **FN** বেশি ভয়ংকর — রোগ ধরা না পড়লে রোগী মারা যেতে পারে।
- Spam filter এ **FP** বেশি সমস্যা — important email spam হয়ে গেলে সমস্যা।

Business এ different type এর ভুলের আলাদা **cost** আছে — Confusion Matrix সেটা বুঝতে সাহায্য করে।

২. F1-Score — Precision আর Recall এর Balance

প্রথমে দুটো জিনিস বুঝতে হবে:

Precision (নির্ভুলতা):

Model যতবার "Positive" বলেছে, তার মধ্যে কতটা সত্যি ছিল?

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

উদাহরণ: Model ১০০ জনকে "cancer আছে" বলল, কিন্তু আসলে মাত্র ৭০ জনের আছে। $\text{Precision} = 70/100 = 0.70$

Recall (সংবেদনশীলতা / Sensitivity):

আসলে যত Positive ছিল, model তার কতটা ধরতে পেরেছে?

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

উদাহরণ: আসলে ১০০ জনের cancer আছে, model ধরেছে ৮০ জনকে। $\text{Recall} = 80/100 = 0.80$

Accuracy কেন ব্যর্থ হয়?

ধরো ১০০০ জন patient এর মধ্যে মাত্র ১০ জনের cancer আছে। তোমার model সবাইকে "cancer নেই" বলে দিল।

$$\text{Accuracy} = 990/1000 = 99\% \text{ 🎉}$$

কিন্তু model আসলে সম্পূর্ণ অকেজো — ১০ জন রোগীকেই miss করেছে!

এই সমস্যার সমাধান: **F1-Score**

F1-Score কী?

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

এটা Precision আর Recall এর **Harmonic Mean**।

Harmonic Mean কেন? Arithmetic Mean কেন না?

ধরো একটা model এর:

- Precision = 1.0 (perfect)
- Recall = 0.0 (সব positive miss করেছে)

Arithmetic Mean = $(1.0 + 0.0) / 2 = 0.5$ ← মনে হচ্ছে মোটামুটি ভালো!

Harmonic Mean (F1) = $2 \times (1.0 \times 0.0) / (1.0 + 0.0) = 0.0$ ← সত্যিকারের অবস্থা দেখাচ্ছে!

👉 **Harmonic Mean** কঠোর — যদি Precision বা Recall এর যেকোনো একটা খুব খারাপ হয়, F1 সেটাকে heavily penalize করে।

F1 তখনই ভালো হবে যখন দুটোই ভালো থাকবে।

৩. ROC Curve — সব Threshold এ Model দেখা

Threshold কী?

Model সাধারণত একটা **probability** দেয়, যেমন: "এই রোগীর cancer হওয়ার সম্ভাবনা 0.73"

তারপর আমরা একটা **threshold** ঠিক করি — যেমন 0.5:

- $0.73 > 0.5 \rightarrow$ "Positive" বলব
- $0.3 < 0.5 \rightarrow$ "Negative" বলব

কিন্তু 0.5 সবসময় সেরা threshold না! **ROC Curve** দেখায় সব সম্ভব threshold এ model কেমন কাজ করে।

ROC Curve কীভাবে তৈরি হয়?

Y-axis: True Positive Rate (TPR) = Recall

$$TPR = TP / (TP + FN)$$

আসল Positive গুলোর মধ্যে কতটা ধরতে পেরেছি।

X-axis: False Positive Rate (FPR)

$$FPR = FP / (FP + TN)$$

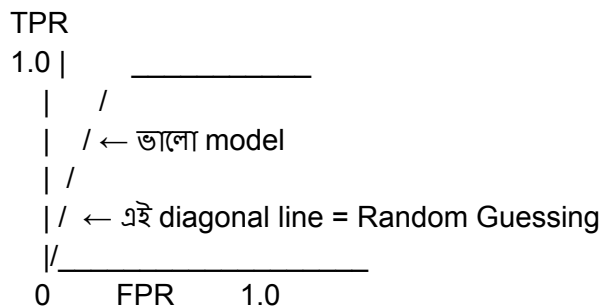
আসল Negative গুলোর মধ্যে কতটাকে ভুলভাবে Positive বলেছি।

Threshold কমালে কী হয়?

Threshold কমালে (যেমন $0.5 \rightarrow 0.3$):

- Model বেশি বেশি "Positive" বলবে
- ☒ বেশি রোগী ধরা পড়বে (TPR বাড়বে)
- ☒ কিন্তু সুস্থ মানুষকেও "রোগী" বলবে (FPR ও বাড়বে)

এই **tradeoff** ই ROC Curve এ ফুটে ওঠে।



একটা ভালো model এর curve উপরের বাম কোণের দিকে থাকবে। Random guess এর curve হবে **45° diagonal**।

8. AUC — ROC কে একটা সংখ্যায় প্রকাশ

AUC = Area Under the ROC Curve

ROC Curve এর নিচের জায়গাটুকু হলো AUC। এটা **0** থেকে **1** এর মধ্যে থাকে।

সহজ **Statistical** মানে:

যদি randomly একজন Positive আর একজন Negative patient নেওয়া হয়, AUC হলো সেই **probability** যে model Positive patient কে বেশি score দেবে।

AUC Score	মানে কী
1.0	Perfect model — দুটো class সম্পূর্ণ আলাদা করতে পারে
0.8 – 0.9	Excellent — খুব ভালো discrimination
0.7 – 0.8	Good

- 0.5 Random guessing এর মতোই — অকেজো
- < 0.5 Random এর চেয়েও খারাপ (predictions উল্টো করলেই ভালো!)

AUC এর সুবিধা:

- **Threshold-invariant** — কোনো নির্দিষ্ট threshold ধরে নেয় না
 - **Scale-invariant** — exact probability values না, ranking টা দেখে
 - Imbalanced dataset এও কাজ করে
-

৫. Cross-Validation — Evaluation কে নির্ভরযোগ্য করা

সমস্যাটা কী?

ধরো তুমি data কে 80% train, 20% test এ ভাগ করলে।

কিন্তু যদি ঐ 20% test data টা **accidentally** সহজ হয়ে যায়? তাহলে model অনেক ভালো দেখাবে — কিন্তু সেটা সত্যি না।

আবার যদি test data কঠিন হয়? তাহলে model খারাপ দেখাবে — কিন্তু সেটাও সত্যি না।

একটা মাত্র split এর উপর ভরসা করা ভাগ্যের উপর নির্ভর করা।

সমাধান: K-Fold Cross-Validation

Data কে **K** টা সমান অংশ (**fold**) এ ভাগ করো। তারপর K বার train/test করো, প্রতিবার একটা আলাদা fold test এ রাখো।

উদাহরণ (**K=5**):

Iteration 1: [TEST] [train] [train] [train] [train]
Iteration 2: [train] [TEST] [train] [train] [train]
Iteration 3: [train] [train] [TEST] [train] [train]
Iteration 4: [train] [train] [train] [TEST] [train]
Iteration 5: [train] [train] [train] [train] [TEST]

প্রতিটা iteration এ একটা score পাবে। Final result = এই ৫টা **score** এর গড় (**mean**) $\pm$ **standard deviation**

কেন এটা ভালো?

- প্রতিটা data point একবার **test** এ আসে
- কোনো একটা "lucky" বা "unlucky" split এর উপর নির্ভর করতে হয় না
- **Standard deviation** বলে দেয় model কতটা consistent — কম SD মানে reliable model

সব **Concept** এর মধ্যে সম্পর্ক

Data



Model Train করো



Prediction করো



Confusion Matrix → TP, TN, FP, FN বের করো



└─ Precision, Recall → F1-Score (fixed threshold এ performance)

└─ বিভিন্ন threshold এ → ROC Curve → AUC (overall performance)



এই পুরো process টা K বার করো → Cross-Validation → Reliable Score
